

Enhancing NeuroEvolution-Based Game Testing: A Branch Coverage Approach for Scratch Programs

Khizra Sohail^{1,2}, Atif Aftab Ahmed Jilani¹, and Nigar Azhar Butt¹

¹Department of Software Engineering, National University of Computer and Emerging Sciences, Islamabad, Pakistan

Corresponding author:
Khizra Sohail²

Email address: khizrasohail739@gmail.com

ABSTRACT

Automated test generation for game-like programs presents unique challenges due to their non-deterministic behavior and complex control structures. The NEATEST framework has been used for automated testing in Scratch games, employing neuroevolution-based test generation optimized for statement coverage. However, statement coverage alone is often insufficient for fault detection, as it does not guarantee execution of all logical branches. This paper introduces a branch coverage-based fitness function to enhance test effectiveness in automated game testing. We extend NEATEST by integrating a branch fitness function that prioritizes control-dependent branches, guiding the neuroevolution process to maximize branch exploration. To evaluate the effectiveness of this approach, empirical experiments were conducted on 25 Scratch games, comparing Neatest with Statement Coverage (N_{SC}) against Neatest with Branch Coverage (N_{BC}). A mutation analysis was also performed to assess the fault detection capabilities of both techniques. The results demonstrate that N_{BC} achieves higher branch coverage than N_{SC} in 13 out of 25 games, particularly in programs with complex conditional structures. Moreover, N_{BC} achieves a lower false positive rate in mutation testing, making it a more reliable approach for identifying faulty behavior in game programs. These findings confirm that branch coverage-based test generation improves test coverage and fault detection in Scratch programs.

Keywords— Automated Testing, Branch Coverage, NeuroEvolution, Search-Based Software Testing (SBST), NEATEST, Scratch

1 INTRODUCTION

The gaming industry is a rapidly growing sector, with an estimated net worth of US\$ 522.46 billion worldwide in 2025, experiencing an annual growth rate of 7.25% [1]. Games are intentionally designed to be interactive, unpredictable, and engaging, which makes automated testing a challenging task [2]. Unlike traditional software applications, games rely on complex user interactions and randomized events, making it difficult to apply conventional test generation strategies [3]. Even when a test generator successfully identifies a relevant sequence of actions, the randomized nature of game execution makes it nearly impossible to reproduce the same test case in subsequent runs. This non-deterministic behavior complicates test generation, as traditional static test suites cannot adapt to dynamically evolving game states.

Currently, games can be tested using manual testing, semi-automated testing, or fully automated testing. Manual Testing is widely used in the gaming industry but is labor-intensive and relies on human expertise [4], [5], [3]. Semi-Automated Testing combines manual and automated approaches, reducing workload but still

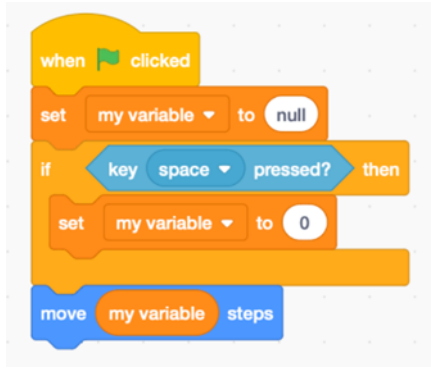


Figure 1. Code Sample from Scratch

requiring human intervention [6], [7], [8]. Automated Testing, in contrast, aims to generate and execute test cases dynamically, offering a scalable and efficient alternative [9], [10], [11]. Traditional automated techniques often either record and replay test scenarios [12] or use recorded test cases as base for evolutionary search [13], but due to the inherent randomness in games, these static test suites fail to provide adequate coverage. Dynamic test suites, which react to unseen game states and adjust test execution in real-time, are necessary to improve the effectiveness of automated testing.

Scratch [14] is a widely used block-based programming language with over 90 million registered users, primarily used for introducing programming concepts to beginners. Many Scratch projects involve game-like elements, providing a learning environment that fosters problem-solving and creativity. Despite its simplicity, Scratch games can contain logical errors that require systematic testing. The Whisker framework [15] offers an execution environment for automated test execution of Scratch programs, supporting both manually written and automatically generated test cases. However, while an execution framework exists, a critical challenge remains: how to generate test suites automatically in a way that ensures adequate test coverage and fault detection.

NEATEST [16] is an automated test suite generator for Scratch games, leveraging NeuroEvolution of Augmenting Topologies (NEAT) [17] to evolve dynamic test agents in the form of neural networks. These neural networks act as test input generators, dynamically interacting with the game environment to reach specific program states. Test generation in NEATEST is by default guided by statement coverage criteria. However, statement coverage alone is one of the weakest coverage criteria, as it does not ensure that all logical branches of a program are tested.

In programs with control structures, execution follows different branches based on decision-making conditions. A fault located in a specific branch may remain undetected if that branch is never executed. NEATEST's statement coverage-based test generation does not guarantee the execution of all logical branches, making it insufficient for uncovering certain types of faults. Consider the example in Figure 1, where a test suite achieves 100% statement coverage but fails to detect a fault because the alternate branch of an if-condition is never triggered. This limitation underscores the need for a more robust coverage strategy that prioritizes branch execution.

Several coverage criteria exist in structural testing, but selecting the appropriate default criterion for an automated test generator is a challenging task. Coverage selection is often based on industry standards or optimized for specific testing goals, such as improving fault detection rates. Branch coverage is widely considered a more effective test coverage metric than statement coverage, as it ensures that both true and false branches of conditional statements are exercised.

For a general-purpose test generator, branch coverage is often preferred because it balances thoroughness and efficiency [18]. It captures logical execution paths, increasing the likelihood of detecting faults in branching logic, without being as computationally expensive as full path coverage[19], [20].

To address the limitations of statement coverage-based testing, this paper introduces a branch coverage-based

fitness function for NEATEST. We define and evaluate two coverage criteria along with their respective fitness functions for guiding NeuroEvolution-based test generation. Our contributions are summarized as follows:

1. We propose a new coverage strategy, (N_{BC}), by developing a branch coverage-based fitness function integrated with NEATEST.
2. We perform an empirical comparison of coverage percentages achieved by Neatest with Branch Coverage (N_{BC}) and Neatest with Statement Coverage (N_{SC}) across 25 Scratch games.
3. We conduct mutation analysis for N_{BC} and N_{SC} , evaluating their fault detection effectiveness using mutation kill score percentages.

This work extends NEATEST by integrating branch coverage-based test generation and demonstrates its effectiveness in improving test coverage and fault detection in Scratch games. Through empirical analysis, we show that branch coverage-based fitness functions enable more efficient and effective test case generation, making them a viable alternative to statement coverage-based approaches.

The remainder of this paper is structured as follows: Section 2 provides the background, detailing key concepts and covers related works. Section 3 describes the proposed methodology. Section 4 outlines the experimental design, research questions, and evaluation metrics. Section 5 presents the results and analysis, discussing the performance of N_{BC} against N_{SC} . Section 6 details threats to validity. Finally, Section 7 concludes the paper by summarizing the contributions and highlighting avenues for future research.

2 BACKGROUND AND LITERATURE REVIEW

This section provides an overview of test coverage criteria (statement and branch coverage) and their significance in search-based software testing (SBST) and neuroevolution-based test generation. Additionally, it discusses prior research related to automated game testing, test generation strategies, and Scratch testing frameworks to position this study within the broader context of existing work.

2.1 Software Testing

Software testing is a critical component in ensuring the quality, reliability, and correctness of software applications. However, it presents several challenges, including ensuring sufficient test coverage, evaluating test effectiveness, detecting critical faults, and maintaining test cases across evolving software versions. One of the most pressing challenges is the evaluation of test quality [21], which is typically assessed using test adequacy criteria [22], metrics that determine whether a test suite sufficiently exercises the software system.

Among these adequacy criteria, statement coverage remains one of the most widely used measures for evaluating test completeness [23]. Test coverage, in particular, ensures that test cases execute different elements of a program's code, helping to uncover defects. Several studies have examined the effectiveness of coverage-based test adequacy criteria in detecting software faults, emphasizing the importance of selecting an appropriate metric for test generation.

This study specifically focuses on two widely used Test coverage criteria: statement coverage and branch coverage.

2.2 Test Coverage Criteria

Test coverage measures the extent to which a test suite exercises a program's control flow. The most commonly used coverage criteria in search-based testing are statement coverage and branch coverage.

2.2.1 Statement Coverage

A statement is the smallest executable unit of a program, ranging from single-line instructions to multi-line compound statements. Achieving 100% statement coverage (SC) implies that every executable statement in the program has been executed at least once [21]. SC is a metric used to measure the degree to which the source code of a program is executed when a particular test suite is run in terms of the number of lines of code (LOC) executed versus total LOC as shown in equation 1.

$$SC = \frac{\text{LOC executed}}{\text{Total LOC}} \times 100\% \quad (1)$$

Statement coverage is one of the simplest and most commonly used test adequacy criteria, particularly in industry applications. However, it does not guarantee comprehensive fault detection since it does not consider whether all control flow paths have been executed. Prior research has identified statement coverage as one of the weakest test coverage metrics, as it may leave critical logic paths untested [23], [22]. Despite its limitations, it remains widely used due to its ease of implementation and computational efficiency.

2.2.2 Branch Coverage

Branch coverage is a stronger structural testing criterion that ensures all control flow branches (i.e., both true and false conditions of decision statements) are executed at least once during testing. It is measured as the ratio of executed branches to total branches in the program (as shown in equation 2), providing a more thorough assessment of test adequacy compared to statement coverage [21].

$$BC = \frac{\text{Branches executed}}{\text{Total Branches}} \times 100\% \quad (2)$$

Since branch coverage inherently subsumes statement coverage, achieving 100% branch coverage guarantees 100% statement coverage, but not vice versa. Several studies have demonstrated that branch coverage-based testing improves fault detection rates, as it ensures that decision points within the code are properly evaluated [23], [22].

Consider the pseudo-code example (Listing 1): If a test input $BAL = 6000$ is used, 100% statement coverage is achieved, but only 50% branch coverage, as the false branch of the if-condition is never executed. A hidden fault (returning a NULL reference) remains undetected unless a second test case, such as $BAL = 4000$, is executed. This second test case enables full branch coverage, ensuring that both execution paths are tested. This example highlights why branch coverage is a superior metric for structural testing.

```

1 Function BankAcc OpenAcc(int BAL)
2     If (BAL >= 5000) Then
3         Return new BankAcc(BAL)
4     EndIf
5     Return NULL
6 EndFunction

```

Listing 1. Function OpenAcc: Creates a new bank account if the initial balance is at least 5000. Otherwise, returns NULL.

2.3 Search-Based Software Testing (SBST) for Test Generation

Search-Based Software Testing (SBST) [24] employs meta-heuristic search algorithms to generate test inputs that optimize test coverage. These algorithms iteratively refine test cases based on fitness functions, which measure how well a test reaches specific program states.

2.3.1 Fitness Function in SBST

The effectiveness of an SBST approach depends on the fitness function guiding the search. Two widely used fitness components are:

1. **Approach Level:** Determines the distance between the executed path and the target statement, encouraging the search process to move toward uncovered code [25].
2. **Branch Distance:** Measures how close a test execution is to reaching a target branch, influencing the selection of test inputs [26].

SBST has been successfully applied in test case generation for method call sequences[27],[28], [29], REST API testing [30], and GUI testing [31],[32], [33] . However, it has limitations when applied to game-like environments, where static test inputs often fail to account for randomized behavior.

2.4 NeuroEvolution-Based Test Generation

To overcome the static nature of SBST-generated test inputs, researchers have explored neuroevolution-based test generation, which evolves dynamic test agents in the form of neural networks.

Neuroevolution has been applied in multiple domains, including game AI testing and software testing automation.

2.4.1 NEATEST Framework

NEATEST [16] extends the Whisker framework [15] to support dynamic test generation for Scratch programs. It uses NeuroEvolution of Augmenting Topologies (NEAT) [17] to evolve neural networks that act as adaptive test input generators, enabling robust testing despite randomized behaviors in games.

Neatest optimizes for statement coverage by default, prioritizing the sequential execution of program statements. Target statements are selected using a Control Dependence Graph (CDG) [34], allowing the search to progress systematically through the program. Fitness functions are used to guide the evolution of neural networks, incorporating both branch distance and approach level.

2.4.2 Neural Networks as Test Oracles

In NEATEST [16], neural networks function as test oracles by evaluating program execution and detecting erroneous behavior. During test execution, the network monitors game states and responses to determine whether observed behavior aligns with expected execution patterns. NEATEST uses node activations and structural changes in the evolved network to assess deviations.

If a neural network exhibits unexpected activation patterns when interacting with a mutated version of the program, the mutation is flagged as a fault. Additionally, large deviations in network structure or activation values indicate that the tested program behaves differently than expected, helping to distinguish between genuine faults and normal randomized variations in execution.

2.4.3 Adapting NEATEST for Branch Coverage

While previous work on NEATEST has focused on statement coverage, this study introduces a branch coverage-based fitness function to improve fault detection capabilities. By modifying the fitness function, we enable NEATEST to prioritize branch execution instead of merely maximizing statement coverage. Hence, test diversity can be improved by ensuring both true and false branches of decision statements are executed. This enhances fault detection rates, as uncovered branches may contain hidden faults.

2.5 Testing Scratch Programs

Scratch is a beginner-friendly, block-based programming language that allows users to create interactive games, animations, and stories by assembling visual code blocks. Each project features a stage (background) and sprites (characters or objects), whose behaviors are controlled through block-based scripts. While the visual blocks enforce syntactic correctness, functional errors can still arise, making testing essential. To address this, Whisker automates testing by providing inputs to Scratch programs and checking their behavior against predefined specifications. Whisker offers two types of test harnesses: manually written and automatically generated. The former involves creating test cases by hand, specifying inputs and expected outcomes. The latter generates diverse test inputs to explore different program states and identify potential issues. Previously Automated test generation for Scratch has been explored through various techniques. One early approach models test generation as a search problem using grammatical evolution [35]. While effective, it is limited by the slow execution of UI-driven tests and the complexity of instrumenting Scratch's virtual machine for fitness evaluation. Later, WHISKER was extended with model-based testing, allowing users to define state diagrams from which test cases could be automatically derived [36]. However, abstract state definition remains a challenge due to its reliance

on domain-specific knowledge, which affects model accuracy and completeness. Other efforts incorporated neuroevolution with search-based testing to adaptively generate test cases [16]. Although promising, this method incurred high training time for the neural networks that act as input generators. To mitigate this, subsequent work leveraged human gameplay traces and gradient descent to accelerate training [13]. To further improve coverage, random and search-based test generation techniques have been employed [2]. However, conventional search techniques still struggle to reach advanced program states, particularly in game like projects. Afterwards, novelty search was introduced to reward behavioral diversity during test generation, helping avoid deceptive fitness landscapes where traditional methods often stagnate [37]. While this approach broadens exploration, it introduces challenges in balancing novelty with goal-directed coverage. Most recently, the NEATEST framework was extended with many-objective optimization algorithms such as MIO and MOSA [38]. While their work incorporates branch coverage as an evaluation metric, it does so only for benchmarking purposes, without elaborating on how the metric is computed or analyzed. Their primary focus lies in enhancing the optimization strategy itself. In contrast, this study explicitly targets and evaluates branch coverage within the context of Scratch. Our work introduces structural refinements for selecting coverage targets using control dependence graphs (CDGs), enabling more precise targeting of branching logic. By focusing on how branch coverage is measured and improved in this domain, this study offers a deeper analysis of the metric's effectiveness and reliability in test generation for Scratch programs.

3 METHODOLOGY: ACHIEVING BRANCH COVERAGE IN SCRATCH

This section describes how branch coverage is computed in Scratch programs, the enhancements made to NEATEST, and the fitness function used for optimizing neural networks in test generation. The methodology includes three key steps:

1. Identification and classification of control structures in Scratch.
2. Optimizing NEATEST to prioritize branch coverage.
3. Designing a fitness function to guide search-based software testing effectively.

3.1 Classification of Control Structures in Scratch

To effectively compute branch coverage in Scratch, it is essential to categorize control structures based on their execution behavior. Some control structures explicitly introduce decision points, while others influence execution flow without creating a distinct false branch. Understanding these classifications allows for better test generation, ensuring that all relevant execution paths are exercised.

Scratch control structures can be categorized into three main types:

1. Branching Blocks, which introduce explicit decision-making.
2. No-False-Branch Blocks, which influence execution without alternative conditions.
3. Execution-Halting Blocks, which pause or stop execution and indirectly affect control flow.

To systematically identify and classify these structures, we define a function called `ControlFilter`. `ControlFilter` analyzes Scratch programs, detects conditional statements, loops, and execution-halting blocks, and categorizes them based on their execution properties. This ensures that the test generation process accounts for all significant control flow elements.

3.1.1 Branching Blocks

Branching blocks introduce conditional execution paths in a Scratch program. These blocks either execute conditionally based on a given expression or introduce loops with exit conditions.

Single-Branch Blocks Single-branch blocks have only one execution path and do not provide an alternative path if the condition is false. These blocks execute only when their condition is satisfied; otherwise, execution proceeds to the next statement. The following Scratch blocks fall under this category:

- **control_if:** Executes a block of code only if the specified condition evaluates to **true**. If the condition is false, execution skips to the next statement. Since this block lacks an explicit false branch, a test suite optimized only for statement coverage may never test the false condition, leading to incomplete coverage.
- **control_repeat, control_repeat_until, and control_forever:** These loops terminate only when specific conditions are met. The `repeat until` loop introduces a **true/false decision**, requiring test cases that cover execution before the condition is met, and execution after the condition is met (loop exit).
- **control_wait_until:** Halts execution until a condition is satisfied. Since execution stalls until the condition is satisfied, testing strategies must ensure cases where the condition is true immediately, and situation where the condition remains false for an extended period.

Double-Branch Blocks Double-branch blocks explicitly introduce two execution paths—a **true** path and a **false** path. To satisfy branch coverage, test cases must be generated to exercise both branches. The primary double-branch block in Scratch is:

- **control_if_else:** Evaluates a condition and executes either the **true branch** or the **false branch**. Both execution paths must be tested for full coverage. If only one branch executes, faults in the untested branch remain undetected.

3.1.2 No-False-Branch Blocks

Some control structures in Scratch do not introduce an explicit false path but still influence execution flow. These blocks execute unconditionally or pause execution temporarily. The main no-false-branch blocks in Scratch include:

- **control_forever:** Executes indefinitely without a terminating condition.
- **control_wait_until:** Pauses execution until a condition is met.

Some other blocks that pause execution temporarily include `wait`, `say`, and `play sound`. These blocks do not introduce explicit **true/false decisions**, but they influence execution flow and timing. Testing strategies targeting these blocks must account for scenarios where:

- The loop runs indefinitely, preventing further execution.
- Execution is paused for a delay, affecting test scheduling.

3.1.3 Execution-Halting Blocks

Execution-halting blocks pause or terminate execution, implicitly affecting program flow. While they do not create explicit branching conditions, they may prevent alternate execution paths from being reached.

The main execution-halting blocks include:

- **control_stop:** Immediately terminates execution.
- **control_wait:** Delays execution for a set duration.

If a **stop block** is reached in a true branch, the false branch may remain untested. Test cases must ensure that:

- Execution is forced beyond the stop block.
- Alternate paths that avoid stopping the script are explored.

3.1.4 Role of ControlFilter in Test Generation

To ensure comprehensive branch coverage, we employ `ControlFilter` to systematically identify and classify control structures. `ControlFilter` first detects all control blocks within the program by filtering opcodes that start with `control_`. It then categorizes them into branching blocks, no-false-branch blocks, and execution-halting blocks. By distinguishing between these types, `ControlFilter` enables the test generator to target conditional paths more effectively, ensuring that all relevant execution states are evaluated.

3.2 Neatest with Statement Coverage (N_{SC})

NEATEST is a neuroevolution-based test suite generator designed for Scratch games. It evolves neural networks as test agents, optimizing them to interact with a Scratch program and generate test cases that maximize coverage. By default, NEATEST optimizes statement coverage, ensuring that every statement in the program is executed at least once [16].

Neatest with Statement Coverage (N_{SC}) is a test generation strategy designed to maximize statement coverage in Scratch programs. By evolving neural networks as test agents, N_{SC} ensures that each executable statement in a Scratch program is covered at least once. However, while this approach is effective in reaching individual program statements, it does not explicitly account for the execution of different branching conditions, leading to potential gaps in the evaluation of program logic.

N_{SC} operates by selecting uncovered statements as test targets. The process begins with the identification of the next uncovered statement using the Control Dependence Graph (CDG), after which a population of neural networks evolves towards executing that statement. Each network is assigned a fitness score based on its effectiveness in reaching the desired statement, and once coverage is achieved, the test generator moves on to the next uncovered statement. This process continues iteratively until all statements in the program have been executed at least once. While this approach provides a measure of completeness, it does not ensure that the logical decisions governing program flow are thoroughly tested.

The primary limitation of N_{SC} lies in its inability to guarantee branch coverage, particularly in cases where decision structures introduce alternative execution paths. Consider the Scratch program shown in Figure 1, which consists of a variable initialization, a conditional check for key input, a conditional assignment, and a movement command. If a test case generated by N_{SC} executes the if-condition where the space key is pressed, the statement inside the condition (`set my variable to 0`) will be covered, along with the subsequent movement command (`move my variable steps`). However, N_{SC} does not ensure that the alternative execution path, where the space key is not pressed, is ever tested. In this scenario, if the variable remains `null`, executing the move command might lead to an error or unexpected behavior. Since N_{SC} does not systematically generate test cases that force the program into both possible execution paths, such issues may go undetected.

This limitation becomes critical when testing programs with multiple branching conditions, as N_{SC} may inadvertently favor one execution path while neglecting others. This can result in a false sense of test completeness, where all statements have been executed, yet critical faults in untested branches remain hidden. The inability to enforce execution of all decision-making constructs makes N_{SC} insufficient for rigorous testing, particularly in scenarios where program correctness depends on the behavior of conditional logic.

To address these shortcomings, Neatest with Branch Coverage (N_{BC}) extends N_{SC} by explicitly prioritizing branch coverage as an optimization goal. Unlike N_{SC} , which merely ensures execution of individual statements, N_{BC} guarantees that both the true and false branches of every conditional statement are exercised. This enhancement not only improves test completeness but also strengthens fault detection by ensuring that all decision-based execution paths in a Scratch program are systematically explored. The following section describes how N_{BC} overcomes the limitations of N_{SC} by integrating a structured approach for prioritizing uncovered branches during test generation.

3.3 Neatest with Branch Coverage (N_{BC})

Neatest with Branch Coverage (N_{BC}) extends N_{SC} by ensuring that both true and false branches of conditional statements are executed. Unlike N_{SC} , which focuses solely on executing statements, N_{BC} explicitly targets

unexecuted branches to provide a more comprehensive evaluation of program logic. This improvement allows N_{BC} to identify faults that N_{SC} may overlook, particularly those related to unexplored execution paths in decision-making structures.

N_{BC} modifies NEATEST's optimization strategy by prioritizing branch execution over statement execution. Instead of selecting uncovered statements as test targets, N_{BC} selects uncovered branches using the Control Dependence Graph (CDG). The core process of N_{BC} is detailed in Algorithm 1, which dynamically selects branches, evolves test agents using neuroevolution, and validates their robustness before including them in the final test suite.

3.3.1 Overview

The Neatest with Branch Coverage (N_{BC}) algorithm (Algorithm 1) extends the Neatest with Statement Coverage (N_{SC}) approach by systematically ensuring that both true and false branches of conditional statements are executed during test generation. Unlike N_{SC} , which focuses on reaching uncovered statements, N_{BC} dynamically prioritizes unexecuted branches as test targets to improve logical path exploration and fault detection.

The algorithm begins by initializing a `requireNextBranch` flag (line 2) to indicate whether a new target branch should be selected. The execution proceeds in an iterative loop (line 3), terminating only when the stopping condition is met (e.g., reaching a time limit or achieving full branch coverage). If a new target branch is required (line 4), the `selectTargetBranch()` function (line 5) is called. This function queries the Control Dependence Graph (CDG) to identify an unexecuted branch that should be selected for testing. The `ControlFilter` module further ensures that control structures are correctly categorized and analyzed to facilitate optimal branch selection.

Once the target branch is determined, a population of neural networks is generated (line 6) using `generatePopulation()`. These networks serve as candidate solutions, evolving through neuroevolution to trigger the selected branch. The algorithm then enters a training phase, where each network executes the program (line 9) and computes a fitness score (line 10) based on its success in executing the target branch.

If a network successfully covers the target branch (line 11), it undergoes a robustness check (line 12) via the `robustnessCheck()` function. This step ensures that the network can reliably execute the same branch across multiple program executions under different conditions. The network's fitness is updated accordingly (line 13), and if it meets the desired robustness count (line 15), it is added to the dynamic test suite (line 16). The branch is then marked as covered in B_u (line 17), and the algorithm sets the flag to `requireNextBranch = true` (line 18), triggering selection of the next uncovered branch.

If no network has successfully covered the branch, the NEAT evolutionary process (line 21) is triggered to mutate and refine the population for improved performance in subsequent iterations. This cycle repeats until all branches are covered or the search budget is exhausted.

The branch selection process (line 5) is further detailed in Section 3.3.2, where the selection strategy ensures that execution history and structural dependencies are considered when selecting the next target branch. This strategic prioritization allows N_{BC} to intelligently shift focus between branches, ensuring comprehensive branch coverage while minimizing redundant test executions.

By integrating `ControlFilter` and refining the branch selection strategy, N_{BC} achieves a more structured and efficient test evolution compared to N_{SC} . The proceeding section details the fitness function (Section 3.3.3), which quantifies how well a test case triggers a given branch and guides neuroevolution toward optimal solutions.

3.3.2 Branch Selection Strategy

A crucial step in N_{BC} is the selection of the next uncovered branch to be tested (line 5 in Algorithm 1). Instead of selecting branches randomly, N_{BC} employs a guided selection approach using `ControlFilter`, which continuously analyzes the Control Dependence Graph (CDG) to determine which branches remain unexecuted.

`ControlFilter` plays a pivotal role in ensuring that test generation is both systematic and efficient. Its primary functions include:

1. **Filtering Out Non-Branching Structures:** Scratch programs contain loops and control blocks that do

Algorithm 1 Neatest with Branch Coverage (N_{BC})

Require: Control dependence graph CDG
Require: List of uncovered branches B_u
Require: Desired robustness count r_d
Ensure: Dynamic test suite D

```
1: function NEATEST( $CDG, B, r_d$ )
2:   requireNextBranch  $\leftarrow$  true
3:   while stopping condition not reached do
4:     if requireNextBranch then
5:        $b_t \leftarrow$  selectTargetBranch( $CDG, B_u$ )
6:        $P \leftarrow$  generatePopulation( $D$ )
7:       requireNextBranch  $\leftarrow$  false
8:     end if
9:     for all network  $n \in P$  do
10:      initiatePlayLoop( $n$ )
11:       $f \leftarrow$  calculateFitness( $n$ )
12:      if  $f == 1$  then  $\triangleright$  Branch was covered
13:         $r_c \leftarrow$  robustnessCheck( $n, r_d$ )
14:         $f \leftarrow f + r_c$ 
15:      end if
16:      network.fitness  $\leftarrow f$ 
17:      if  $f == r_d$  then
18:         $D \leftarrow D + n$ 
19:         $B_u \leftarrow B_u - b_t$ 
20:        requireNextBranch  $\leftarrow$  true
21:      end if
22:    end for
23:    if not requireNextBranch then
24:       $P \leftarrow$  NEAT( $P$ )
25:    end if
26:  end while
27:  return  $D$ 
28: end function
```

not contribute to branch coverage. ControlFilter ensures that the test generation process focuses only on relevant conditional branches.

2. **Tracking Uncovered Branches:** ControlFilter maintains a list of branches that have not been executed yet, ensuring that these branches are targeted before redundant executions.
3. **Guiding Selection Strategy:** Rather than selecting branches arbitrarily, ControlFilter prioritizes execution paths based on two key factors:
 - **Execution History:** If a branch has already been executed at least once, N_{BC} shifts focus to an unexecuted branch within the program. This prevents redundant test executions and ensures systematic exploration of new logic paths.
 - **Structural Dependencies:** If a branch leads to deeper, unexplored code segments, it is prioritized over branches that lead to already-covered regions. This ensures that N_{BC} progressively explores the program structure rather than stagnating on easy-to-reach branches.

For example, in a Scratch game where a character collects different items (e.g., coins and apples), the initial test case might focus on collecting coins. If, during evolution, the same test incidentally collects apples, N_{BC}

then prioritizes testing for apple collection in future iterations. By doing so, N_{BC} ensures that all branches are executed intentionally, rather than relying on incidental test coverage.

ControlFilter enhances branch selection by refining the search space, directing the test generator toward meaningful execution paths, and reducing wasted test efforts. Without ControlFilter, N_{BC} could end up exploring execution paths inefficiently, leading to longer optimization times and incomplete branch coverage.

3.3.3 Fitness Evaluation

In Neatest with Branch Coverage (N_{BC}), neural networks evolve to maximize their ability to execute both branches of conditional statements. To guide this evolution, each network is evaluated using a fitness function that quantifies its effectiveness in reaching and executing an uncovered branch. The fitness score is based on two fundamental factors:

1. **Approach Level** (α) – Represents how far execution is from reaching the control dependency of the target branch.
2. **Branch Distance** (β) – Measures how much execution deviates from taking the desired branch once the control statement is reached.

The fitness function integrates these two metrics to prioritize networks that are closest to executing an untested branch. Networks with lower approach levels and smaller branch distances receive higher fitness scores, increasing their likelihood of being selected for further evolution. Since both α and β can take on large values, they are normalized to ensure fair contribution to the fitness score. We define the normalization function as follows:

$$\eta(x) = \frac{x}{1+x} \quad (3)$$

This function ensures that all values are scaled into the range $(0, 1]$, preventing excessive influence from large values. The fitness score for a neural network is computed using the normalized approach level and branch distance:

$$f = \eta(\alpha + \eta(\beta)) \quad (4)$$

- α (Approach Level): Determines the number of control dependencies between the execution point and the target branch. If $\alpha = 0$, the execution has already reached the control statement.
- β (Branch Distance): Represents how close the execution is to taking the desired branch. A smaller β means the network is closer to executing the correct branch and $\beta = 0$ means that the branch has been executed.

If the fitness function results in $f = 0$, it means that the target branch has been successfully executed. At this point, robustness testing is performed to verify whether the network can reliably execute the same branch across multiple randomized test runs. If the network passes robustness testing, it is added to the final dynamic test suite, ensuring stable test generation.

```

1 When green flag clicked
2   If (X > 0)
3     If (Y = 0)
4       Say "C"
5     Else
6       Say "B"
7   Else
8     Say "A"
```

Listing 2. Code Sample 2.

Computing Fitness Score: Example Walkthrough

To effectively evolve test cases that maximize branch coverage, N_{BC} relies on branch distance as a key component of its fitness function. Branch distance quantifies how far an execution path is from taking the desired branch in a conditional statement. The smaller the branch distance, the closer the neural network is to triggering the correct execution path.

Branch distance is computed differently depending on whether the approach level is satisfied. If the approach level is greater than zero, meaning the execution has not yet reached the control dependency of the target branch, then branch distance is calculated for the nearest control dependency rather than the branch itself. If the approach level is zero, meaning the execution has reached the target control statement, branch distance measures how close the execution is to taking the correct branch.

Consider the Scratch program represented in Listing 2 and its corresponding CFG shown in Figure 2a. This program consists of nested conditional statements where the execution follows different branches based on the values of X and Y . The objective is to compute the branch distance required to execute a specific branch.

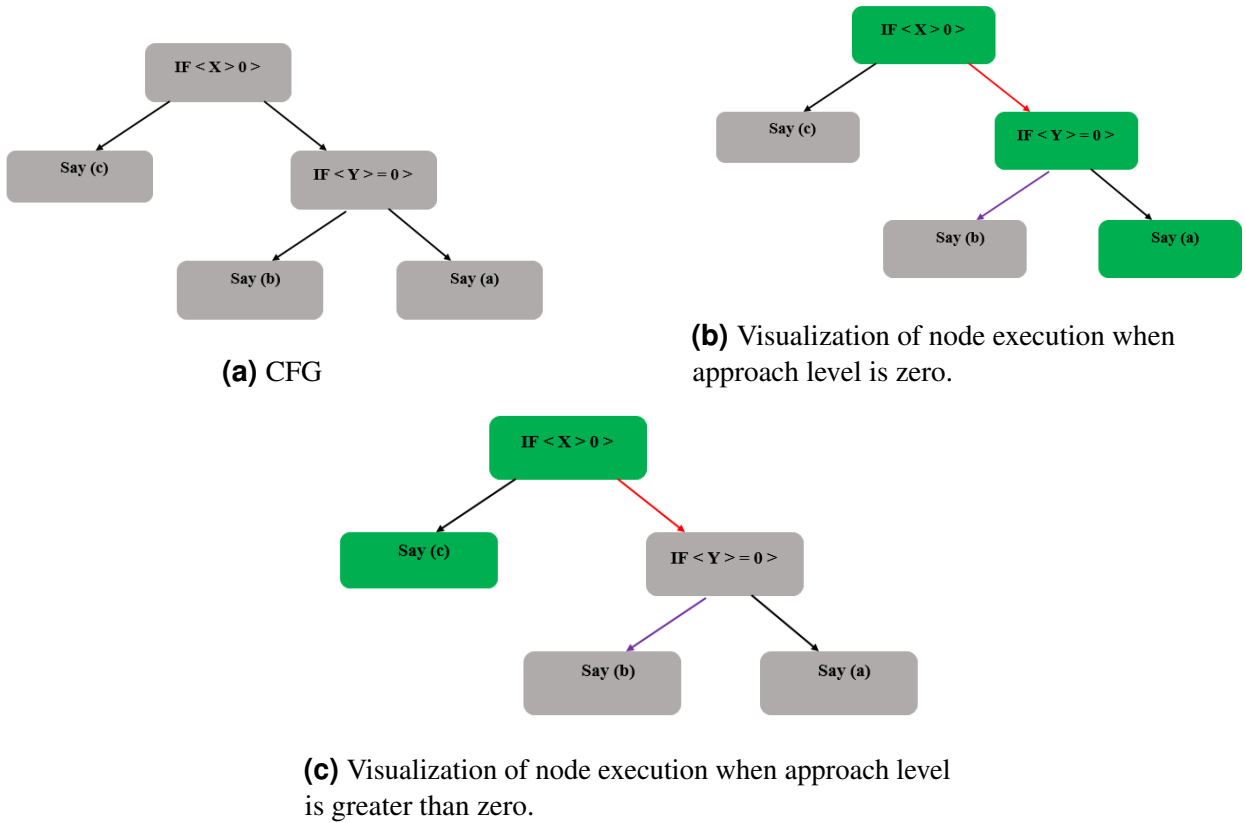


Figure 2. CFG corresponding to Listing 2.

Case 1: Approach Level = 0: Suppose the goal is to execute the false branch of `If (Y = 0)` and the input values are $X = 10$, $Y = 1$. As the execution reaches the condition `If (Y = 0)`, it evaluates to false, leading to the execution of the true branch (`Say (B)`) instead of the target false branch (`Say (C)`) as shown on Figure 2b.

The branch distance in this case is calculated as:

$$\beta = |Y - 0| = |1 - 0| = 1$$

This indicates that reducing Y to zero would have resulted in correct execution.

Case 2: Approach Level > 0: Consider an alternative input where $X = -10$, $Y = -1$. Since $X \leq 0$, execution follows the false branch of `If (X > 0)`, leading to `Say (A)` and bypassing `If (Y = 0)` entirely as shown in Figure 2c.

In this case, the branch distance is calculated relative to the closest control dependency.

Since execution never reached the condition `If (Y = 0)`, the approach level is determined first. The execution must first satisfy `If (X > 0)` before it can reach `If (Y = 0)`, making the approach level:

$$\alpha = 1$$

The branch distance for the unmet condition `If (X > 0)` is calculated as:

$$\beta = |X - 0| = |-10 - 0| = 10$$

This indicates that increasing X to be greater than zero would allow execution to progress toward the next conditional branch.

Fitness Calculation: Once the approach level and branch distance are computed, the fitness function (Equation 4) is applied.

For **Case 1** ($X=10$, $Y=1$), where approach level is zero and branch distance is 1, fitness is calculated as follows:

$$f = \eta\left(0 + \eta(1)\right) = \eta\left(0 + \frac{1}{1+1}\right) = \eta(0.5) = \frac{1}{1+0.5} = 0.666$$

For **Case 2** ($X=-10$, $Y=-1$), where the approach level is one and the branch distance is ten, fitness is calculated as follows:

$$f = \eta\left(1 + \eta(10)\right) = \eta\left(1 + \frac{10}{1+10}\right) = \eta(1.91) = \frac{1.91}{1+1.91} = 0.656$$

The computed fitness score directly influences the evolution of neural networks in N_{BC} . Higher fitness scores indicate that test cases need further improvement. Through mutation and crossover, networks with lower fitness values (closer to zero) are favored in the selection process, driving the neuroevolution towards executing all untested branches. This iterative approach ensures that all logical paths are covered, thereby improving both branch coverage and fault detection capabilities in Scratch programs.

4 EVALUATION SETUP

This section describes the experimental setup used to evaluate the effectiveness of Neatest when optimized for branch coverage (N_{BC}) compared to statement coverage (N_{SC}). The evaluation is structured around two core research questions (RQs), each designed to measure different aspects of test generation performance. To ensure comprehensive and reproducible results, we conduct experiments using a diverse set of Scratch programs, employ statistical significance tests, and analyze the impact of branch-aware test generation on both test coverage and fault detection capabilities.

4.1 Research Questions

To evaluate the effectiveness of the proposed testing approach, we aim to answer the following research questions:

RQ1 How does test coverage differ when NEATEST is optimized for branch coverage (N_{BC}) compared to statement coverage (N_{SC})?

Here, test coverage refers to both statement coverage (SC) and branch coverage (BC). Since N_{SC} is designed to maximize statement coverage, it may achieve partial branch coverage as a side effect. However, N_{BC} explicitly targets branch coverage, ensuring that both true and false branches of conditional statements

are executed. This research question examines whether N_{BC} achieves superior branch coverage while maintaining comparable statement coverage to N_{SC} .

RQ2 How does test oracle effectiveness vary when NEATEST is optimized for branch coverage (N_{BC}) compared to statement coverage (N_{SC})?

While high coverage is crucial, it does not always correlate with the ability to detect program faults. To assess test oracle effectiveness, we measure the mutation score, which represents the percentage of artificially injected faults (mutants) that the generated tests can detect. If N_{BC} provides better branch coverage, we expect it to generate more effective test cases capable of revealing faults in conditional logic.

NEATEST is integrated as part of the Whisker framework and is publicly available as an open-source tool¹. The dataset for our experiments, raw experimental results, and reproduction scripts are also available on GitHub.

4.2 Dataset

We reuse the dataset established in the NEATEST study [16] for evaluation. The dataset comprises 187 Scratch programs collected from beginner’s Scratch programming books², prior studies, and online tutorial websites (Code Club³, Linz Coder Dojo⁴, learnlearn⁵, junilearning⁶). The dataset was curated based on three selection criteria:

1. The program must be a game that accepts user inputs, processes them through the Whisker framework, and presents a challenge to the player.
2. The game must exhibit dynamic behavior, requiring reactive test case generation.
3. Programs that lack a meaningful challenge—where arbitrary input could trivially achieve full coverage—were excluded.

After filtering based on these conditions, the final dataset comprises 25 unique Scratch games, each exhibiting distinct interaction challenges. These games are presented in Table 1.

4.3 Experimental Setup

All experiments were conducted on a dedicated computing cluster consisting of nine nodes, each powered by an AMD EPYC 7443P CPU running at 2.8 GHz. The Whisker framework allows for an acceleration factor to speed up test execution. To ensure efficient testing, we set this factor to infinity (∞), maximizing execution speed.

4.3.1 Configuration for RQ1: Coverage Comparison Between Statement and Branch Coverage

To investigate how test coverage varies when NEATEST is optimized for branch coverage, we extended the framework by introducing a fitness function specifically designed to target branch coverage. This was compared against the existing statement coverage function, which, while focused on maximizing statement coverage, may incidentally cover branches without explicitly optimizing for them. Each experiment was conducted under the following conditions:

- Population size of 150 neural networks for each fitness function.
- Search duration of 5 hours.

¹[August 2022] <https://github.com/se2p/whisker>

²[August 2022] <https://nostarch.com/catalog/scratch>

³[August 2022] <https://projects.raspberrypi.org/en/codeclub>

⁴[August 2022] <https://coderdojo-linz.github.io/uebungsanleitungen/programmieren/scratch/>

⁵[August 2022] <https://learnlearn.uk/scratch/>

⁶[August 2022] <https://junilearning.com/blog/coding-projects/>

Table 1. Overview of the 25 Scratch game projects used for evaluation, detailing the number of sprites, scripts, and executable statements in each.

Projects	#Sprites	#Scripts	#Statements	Projects	#Sprites	#Scripts	#Statements
FlappyParrot	2	7	37	FinalFight	13	48	286
Frogger	8	22	107	OceanCleanup	11	22	156
Fruitcatching	3	4	55	Pong	2	2	15
LineUp	2	4	50	RioShootout	8	26	125
BrainGame	3	18	76	Dragons	6	33	381
EndlessRunner	8	29	163	WhackAMole	10	14	391
Snake	3	14	60	SpaceOdyssey	4	13	116
CatchTheDots	4	10	82	SnowballFight	3	6	39
CatchTheGifts	3	7	68	HackAttack	6	9	93
CityDefender	10	12	97	DieZauberlehrlinge	4	14	87
CatchingApples	2	3	25	Dodgeball	4	10	78
DesertRally	10	27	212	BirdsShooter	4	10	69
FallingStars	4	4	91	Mean	5.5	16.6	118.3

- Non-improving generation limit set to 5.
- Species size of 5.
- Maximum playthrough duration of 5 seconds (equivalent to 100 seconds due to acceleration).

To ensure statistical robustness, we repeated the experiment 30 times ($\mathcal{N} = 30$), following the recommendations of Arcuri et al. [39]. Independent repetitions were performed for each fitness function [39]. The Vargha and Delaney (A12) effect size [40] and the Mann-Whitney U-test [41] ($\alpha = 0.05$) were applied to assess statistical significance.

4.3.2 Configuration for RQ2: Mutation Analysis for Test Oracle Effectiveness

To assess whether neural networks can function effectively as test oracles, we conducted a mutation analysis using eight predefined mutation operators (Table 2) drawn from a widely accepted set of sufficient mutation operators [42]. These operators were applied to all 25 selected Scratch games. A mutant was classified as “killed” under either of the following conditions: (1) the activation values of the neural network exceeded a predefined threshold of 30, based on Likelihood-based Surprise Adequacy (LSA) [16], or (2) the network’s structure underwent a significant change during execution.

For activation-based evaluation, we employed LSA [43], which leverages Kernel Density Estimation (KDE) [44] to assess how unexpected or “surprising” the activations of selected neurons are when the network is presented with mutated inputs.

To ensure robustness and mitigate the influence of randomness inherent to game behavior, each extracted test suite was evaluated across ten different random seeds, resulting in 100 executions per test suite per game. While RQ1 used 30 repetitions to assess coverage, the increased behavioral variability in mutation analysis justifies a higher repetition count. Prior studies have found 100 runs to provide a reliable basis for mutation-based evaluation and to minimize noise from incidental execution shifts [16]. Additionally, to manage the risk of combinatorial explosion in the number of mutant variants, each experiment was constrained to 50 randomly selected mutants per mutation operator per repetition.

4.4 Evaluation Metrics

In this section, we present the definition of the evaluation metrics: statement coverage and decision coverage (RQ1), and, mutation score (RQ2).

Table 2. Overview of the eight mutation operators applied during evaluation. These operators introduce syntactic changes to Scratch programs to simulate faults and assess the fault detection capability of test oracles, as defined in [16].

Mutation Operator	Description
Key Replacement Mutation	Replaces a block key’s listener.
Single Block Deletion	Removes a single block.
Script Deletion Mutation	Deletes all blocks of a given script.
Arithmetic Operator Replacement	Replaces an arithmetic operator.
Logical Operator Replacement	Replaces a logical operator.
Relational Operator Replacement	Replaces a relational operator.
Negate Conditional Mutation	Negates Boolean blocks.
Variable Replacement Mutation	Replaces a variable.

For *RQ1*, we use test coverage in terms of statement coverage percentage (SC) as depicted in equation 1 and branch coverage percentage (BC) as depicted in equation 2 .

Additionally, for *RQ2*, We consider the mutation score (as shown in equation 5) as a criterion to discover whether the test cases have a good bug-revealing ability. It is calculated as the ratio of the number of killed mutants to the total number of mutants.

$$\text{Mutation Score} = \frac{\text{Killed Mutants}}{\text{Total Mutants}} \quad (5)$$

5 RESULTS AND ANALYSIS

5.1 RQ1: Test Coverage

To assess the effectiveness of Neatest with Branch Coverage (N_{BC}) compared to Neatest with Statement Coverage (N_{SC}), we conducted experiments on 25 games in our dataset. The primary objective was to determine whether N_{BC} achieves higher test coverage ratios than N_{SC} .

5.1.1 Branch Coverage

The average branch coverage percentages obtained for N_{SC} and N_{BC} across 30 experimental runs for each game are summarized in Table 3 and shown in figure 3. The results indicate that N_{SC} achieved a mean branch coverage of 84.30%, whereas N_{BC} achieved 86.31%, demonstrating a slight improvement. However, deeper examination reveals interesting trends based on the structural characteristics of individual games.

Case 1: N_{BC} Significantly Outperforms N_{SC}

N_{BC} outperforms N_{SC} significantly in 13 games, with statistically significant p-values, indicating that N_{BC} consistently achieves higher branch coverage (as shown in Table 3). These games include: BrainGame, LineUp, EndlessRunner, HackAttack, CatchTheGifts, DesertRally, OceanCleanup, Whack-A-Mole, SpaceOdyssey, SnowballFight, and DieZauberlehrlinge.

The improved performance of N_{BC} in these games suggests that its ability to force execution of both true and false branches allows for greater structural exploration, resulting in higher coverage ratios.

Case 2: N_{BC} Achieves Higher Mean Coverage, but Not with Statistical Significance

In 5 games, N_{BC} achieved better mean coverage than N_{SC} , but the p-values exceeded 0.05 (as shown in Table 3), indicating high variance in performance across runs. These games are: FlappyParrot, FruitCatching, Snake, CityDefender, and Dragons.

The inconsistency in N_{BC} ’s performance can be attributed to variability in evolved test strategies. Additionally, static analysis of the game structures reveals that several if-conditions are mutually

Table 3. Mean branch coverage% for N_{SC} and N_{BC} for each case study for 30 independent runs along with p-values showing significance and effect size

#	Project	N_{SC}	N_{BC}	P	A12
1	LineUp	90.47	100.00	0.00	0.03
2	BrainGame	98.12	100.00	0.00	0.35
3	EndlessRunner	59.26	66.66	0.00	0.24
4	CatchTheGifts	96.94	100.00	0.00	0.18
5	DesertRally	95.25	95.86	0.00	0.17
6	OceanCleanup	72.04	75.64	0.01	0.31
7	WhackAMole	73.42	77.77	0.00	0.28
8	SpaceOdyssey	90.49	91.21	0.01	0.33
9	SnowballFight	75.94	83.04	0.00	0.09
10	HackAttack	95.52	97.63	0.00	0.28
11	DieZauberlehrlinge	36.77	45.16	0.01	0.31
12	Froger	91.33	90.67	0.02	0.64
13	Dodgeball	91.08	89.68	0.02	0.665
14	FlappyParrot	81.21	82.12	0.08	0.45
15	FruitCatching	90.71	91.78	0.08	0.37
16	Snake	77.85	78.80	0.15	0.43
17	CityDefender	63.87	64.41	0.18	0.45
18	Dragons	49.66	52.75	0.11	0.38
19	CatchingApples	100.00	100.00	1.00	0.5
20	BirdsShooter	100.00	100.00	1.00	0.5
21	RioShootout	94.54	94.54	1.00	0.5
22	CatchTheDots	90.76	89.14	0.06	0.60
23	Pong	100.00	98.88	0.16	0.53
24	FallingStars	94.16	93.78	0.23	0.57
25	FinalFight	98.20	98.15	0.34	0.43
	Mean	84.30	86.31	-	0.3836

exclusive, meaning that achieving one condition automatically negates the other. This reduces the advantage of explicitly optimizing for branch coverage.

For example, in FlappyParrot and Snake, the primary control structures are forever loops and halting blocks, which do not significantly impact branch coverage. In FruitCatching and CityDefender, the most valuable sprites for branch coverage contain if-conditions that are mutually exclusive. In FruitCatching, the bowl sprite has two if-conditions—one to move left, the other to move right. Similarly, the apple sprite has two if-conditions—one checking if it touches the bowl, the other checking if it touches the ground. In CityDefender, the meteorite sprite has two if-conditions—one checking for impact with the city, the other for impact with a color-coded target. In these cases, branch coverage behaves similarly to statement coverage, reducing N_{BC} 's advantage. In the Dragons game, N_{BC} 's search process requires more sophisticated evolution strategies to optimize coverage.

Case 3: N_{BC} and N_{SC} Achieve Equal Coverage

In 3 games, N_{SC} and N_{BC} achieved nearly identical branch coverage percentages (as shown in Table 3), making them functionally equivalent. These games are: CatchingApples, BirdsShooter, and RioShootout.

Static analysis of these games reveals that they have very few control dependencies (i.e., few branch conditions). Hence, the allocated search budget was sufficient for both methods to achieve full coverage. In RioShootout, even though the number of statements and sprites is higher, the key sprite (ball) has two independent if-conditions—one for scoring a goal and one for tracking missed goals. These if-conditions are mutually exclusive, meaning that statement coverage automatically results in branch coverage.

Case 4: N_{SC} Achieves Higher Mean Coverage, but Not with Statistical Significance

In 4 games, N_{SC} achieved slightly higher mean coverage than N_{BC} . These games are: CatchTheDots,

Pong, FallingStars, and FinalFight.

Although the differences in coverage were small (0-2%), this suggests that N_{SC} performs better in games with simpler control dependencies or higher randomness.

Pong has the fewest number of statements and sprites, meaning that full coverage can be achieved efficiently using N_{SC} . FinalFight contains a high number of statements but very few control structures, reducing the difference between N_{SC} and N_{BC} 's performance. Additionally, intrinsic randomness in games like FallingStars means that test generators using N_{SC} may sometimes incidentally trigger more if-conditions than N_{BC} .

Case 5: N_{SC} Significantly Outperforms N_{BC}

In Froger and Dodgeball, N_{SC} attains a slightly higher mean branch coverage than N_{BC} (as shown in Table 3). The p-values (0.02) indicate that N_{SC} 's performance is statistically significant, meaning the observed difference is unlikely due to chance. This can be attributed to the structure of these games, where most control statements are simple and inherently lead to high branch coverage when optimizing for statement coverage. Additionally, certain branches behave like sequential statements, making explicit branch optimization less impactful. Since N_{BC} systematically explores both true and false paths, it may require additional iterations to fully optimize coverage, whereas N_{SC} achieves similar results more efficiently.

Insights from Branch Coverage Analysis

The empirical evaluation of Neatest with Statement Coverage (N_{SC}) and Neatest with Branch Coverage (N_{BC}) demonstrates that optimizing for branch coverage generally improves test coverage, but with some variability depending on game structure. N_{BC} achieves a higher average coverage of 86.31% compared to 84.30% for N_{SC} , indicating an improvement in branch coverage when explicitly optimizing for it. However, the extent of this improvement is influenced by factors such as mutually exclusive if-conditions, game randomness, and the complexity of control structures.

For games with intricate decision logic, N_{BC} provides a clear advantage by systematically exploring both true and false branches, ensuring higher fault exposure. However, in games where branches behave similarly to statements, N_{BC} 's advantage diminishes. Additionally, highly randomized games can introduce inconsistent performance, resulting in cases where N_{SC} occasionally achieves comparable or even slightly better coverage.

Overall, N_{BC} proves to be a more effective test generation strategy when branch coverage is a critical testing criterion, but its effectiveness depends on the structural characteristics of the software under test.

5.1.2 Statement Coverage

The average Statement coverage percentages obtained for N_{SC} and N_{BC} across 30 experimental runs for each game are summarized in Table 4 and shown in figure 3. The results indicate that N_{SC} achieved a mean statement coverage of 90.67%, whereas N_{BC} achieved 91.11%, demonstrating a slight improvement. However, deeper examination reveals interesting trends based on the structural characteristics of individual games and performance of both N_{SC} and N_{BC} in terms of Statistical significance.

Case 1: N_{BC} Significantly Outperforms N_{SC}

N_{BC} significantly outperforms N_{SC} in five games (as shown in Table 4), as indicated by statistically significant p-values ($p < 0.05$). These games include:

The structural complexity of these games explains N_{BC} 's superior performance. EndlessRunner and WhackAMole contain deeply nested control structures (e.g., if statements nested up to 4–5 levels).

Table 4. Mean Statement coverage% for N_{SC} and N_{BC} for each case study for 30 independent runs along with p-values showing significance and effect size

#	Project	Statement Coverage			
		N_{SC}	N_{BC}	p value	A12
1	WhackAMole	75.65	78.65	0.01	0.32
2	SpaceOdyssey	86.90	87.21	0.01	0.32
3	EndlessRunner	74.91	80.83	0.01	0.37
4	OceanCleanup	82.93	85.03	0.03	0.36
5	DieZauberlehrlinge	64.83	68.20	0.04	0.35
6	Dodgeball	94.83	93.68	0.03	0.57
7	Frogger	93.08	93.65	0.04	0.56
8	DesertRally	93.90	93.65	0.04	0.56
9	CityDefender	72.78	75.00	0.03	0.37
10	BirdsShooter	100	100	1.0	0.5
11	CatchTheGifts	100	100	1.0	0.5
12	LineUp	100	100	1.0	0.5
13	BrainGame	100	100	1.0	0.5
14	SnowballFight	95.21	95.21	0.5	0.5
15	CatchingApples	100	100	1.0	0.5
16	RioShootout	100	100	1.0	0.5
17	FlappyParrot	91.35	92.07	0.16	0.46
18	Snake	91.72	94.80	0.15	0.47
19	HackAttack	92.97	93.65	0.25	0.48
20	Dragons	100	100	1.0	0.5
21	CatchTheDots	99.55	98.91	0.25	0.39
22	FallingStars	98.87	97.65	0.3	0.45
23	FinalFight	98.97	98.07	0.5	0.53
24	Pong	98.97	98.07	0.5	0.53
25	Fruitcatching	96.67	96.18	0.75	0.52
	Mean	90.67	91.11	-	0.4675

Since control structures introduce multiple statements, achieving high statement coverage requires triggering them at least once. WhackAMole, SpaceOdyssey, EndlessRunner, OceanCleanup, and DieZauberlehrlinge.

N_{SC} explicitly targets uncovered statements but does not prioritize executing alternative paths within a branching structure. As a result, statements within unexecuted branches may remain uncovered. Conversely, N_{BC} systematically ensures that both paths of a conditional statement (if and else) are exercised, increasing the likelihood of reaching additional statements.

This advantage is evident in OceanCleanup and DieZauberlehrlinge, which contain deeply nested if-else structures (up to six levels), and SpaceOdyssey, where one sprite has three levels of nested conditions. Since N_{SC} does not enforce execution of both sides of a condition, some statements remain uncovered. N_{BC} , by ensuring structured exploration, naturally improves statement coverage.

While N_{BC} consistently achieves higher mean coverage, statistical validation confirms that this improvement is systematic rather than incidental. The Mann-Whitney U test determines whether one approach consistently outperforms the other across multiple runs.

- A low p-value ($p < 0.05$) confirms that N_{BC} 's higher statement coverage is statistically significant.
- Higher mean coverage alone does not imply significance; consistency across experimental runs determines superiority.

For these five games, N_{BC} 's improved statement coverage is reinforced by low p-values, confirming that its structured approach leads to consistent gains.

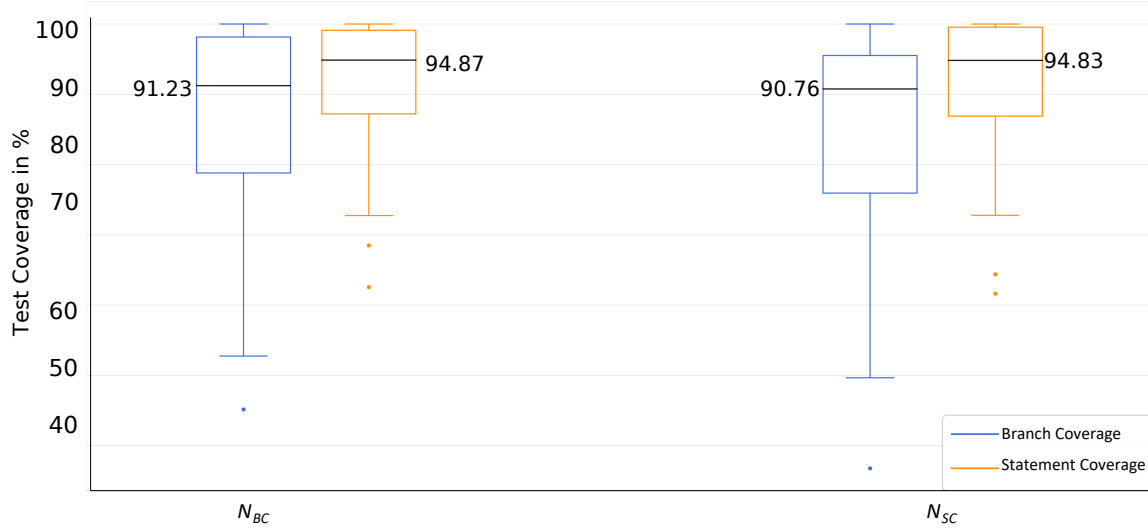


Figure 3. Average Branch and Statement coverage percentages for 30 runs of all case studies combined with respect to N_{BC} and N_{SC}

Case 2: N_{BC} Achieves Higher Mean Coverage, without Statistical Significance

In four games, N_{BC} achieved better mean coverage than N_{SC} (as shown in Table 4), but the p-values exceeded 0.05, indicating high variance in performance across runs. These games are: FlappyParrot, Snake, HackAttack, and Dragons.

The inconsistency in N_{BC} 's performance can be attributed to variability in evolved test strategies. Additionally, static analysis of the game structures reveals that several if-conditions are mutually exclusive, meaning that achieving one condition automatically negates the other. For example, in FlappyParrot and Snake, the primary control structures are forever loops and halting blocks. These blocks are important for achieving statement coverage. Hence, N_{SC} has more reliable performance throughout various runs. In HackAttack, the game structure includes score-dependent and chance-based termination conditions, leading to inconsistent N_{BC} performance. The Virus sprite interacts with Neo-cat to increase the score, but the game halts at a fixed threshold. Additionally, the loss condition depends on a separate counter reducing over time. These constraints limit N_{BC} 's exploration, making its performance highly variable. In the Dragons game, N_{BC} 's search process requires more sophisticated evolution strategies to optimize coverage.

Case 3: N_{BC} and N_{SC} Achieve Equal Coverage

In 7 games, N_{SC} and N_{BC} achieve nearly identical statement coverage (as shown in Table 4). These games are: BirdShooter, CatchTheGifts, LineUp, BrainGame, SnowballFight, CatchingApples, and RioShootout.

This outcome is primarily influenced by the simplicity of the game logic and the presence of structural constraints that limit potential differences between the two approaches. In BirdShooter and CatchingApples, the code structure is straightforward, allowing both approaches to easily achieve full coverage. Prior results from RQ1.1 (Table 3) demonstrated that even branch coverage was maximized in these games, reinforcing the notion that their logic does not impose significant challenges for either technique. For SnowballFight and RioShootout, dead code prevents either method from reaching full statement coverage. Since these unreachable statements remain unexecuted regardless of the testing strategy, both approaches exhibit similar performance. In CatchTheGifts, LineUp, and BrainGame,

while control structures exist, executing just one control path is sufficient to achieve high statement coverage. Unlike games with deeply nested branching logic, these structures do not demand systematic exploration of both paths to maximize coverage. As a result, NSC performs comparably to NBC, despite its probabilistic approach to statement execution.

Case 4: N_{SC} Achieves Higher Mean Coverage, without Statistical Significance

In 5 games, N_{SC} achieved slightly higher mean coverage than N_{BC} (as shown in Table 4). These games are: CatchTheDots, FallingStars, FinalFight, Pong, and FruitCatching.

Although the differences were minimal (ranging from 0% to 1.18%), this trend suggests that N_{SC} performs better in games with fewer and simpler control dependencies or a higher degree of randomness. In CatchTheDots and FruitCatching, the number of statements is low, and the control structures are simple. Many conditional statements in these games, such as if or else-if blocks, are mutually exclusive. This means that triggering one condition (e.g., a true path) inherently implies coverage of the contrasting condition (e.g., a false path). For example, in FruitCatching, if a fruit falls into the bowl (true path), it does not reach the ground (false path), allowing both statements to be covered through a single execution path. Similarly, in CatchTheDots, if a red ball appears and is caught by the red controller, the score increases by one; otherwise, a life is deducted. Since these conditions are mutually exclusive, NSC efficiently covers them. Pong has the lowest number of statements and sprites, making full coverage straightforward for NSC. FinalFight contains a high number of statements but very few control structures, minimizing the advantage of NBC's systematic branching approach. Lastly, in FallingStars, intrinsic randomness can cause NSC to incidentally trigger more if conditions than NBC, contributing to its slightly better performance in statement coverage.

Case 5: N_{SC} Significantly Outperforms N_{BC}

In four games, Dodgeball, Frogger, DesertRally, and CityDefender, N_{SC} attains slightly higher mean statement coverage (as shown in Table 4). The statistical significance of the p-values ($p < 0.05$) confirms that N_{SC} 's superior performance in this case is not due to random variation but a consistent trend across multiple runs.

This can be attributed to the inherent focus of N_{SC} on maximizing executed statements, leading to more efficient coverage in games where most control structures do not significantly alter execution paths. Additionally, the presence of mutually exclusive conditions and simple branching structures may allow N_{SC} to achieve near-complete coverage without the need for extensive exploration. Since N_{BC} prioritizes branch execution, it may allocate resources inefficiently in cases where achieving full statement coverage does not necessarily require optimizing for both true and false branches.

Insights from from Statement Coverage

The comparative analysis of N_{SC} and N_{BC} in statement coverage highlights several key insights. In structurally complex games with deeply nested control structures, such as WhackAMole, SpaceOdyssey, and EndlessRunner, N_{BC} consistently achieves higher statement coverage due to its ability to systematically explore all conditional branches. Conversely, in games with simpler control flows, such as Dodgeball, Frogger, and CityDefender, N_{SC} performs better by maximizing executed statements without the need for exhaustive branch exploration. Notably, in cases where both approaches achieve similar coverage, such as BirdShooter, LineUp, and BrainGame, the game logic is either straightforward or contains dead code, limiting the potential differences between the two techniques. Additionally, in certain games like FlappyParrot and Snake, the presence of mutually exclusive conditions results in high variance for N_{BC} , leading to inconsistent performance. Overall, while N_{BC} demonstrates an advantage in structurally intricate games, N_{SC} remains more efficient in scenarios where statement coverage can

Table 5. Mutation kill score % achieved by N_{SC} and N_{BC}

Projects	#Total mutants generated for N_{SC}	Average Mutation Score N_{SC}	#FP	#Total mutants generated for N_{BC}	Average Mutation Score N_{BC}	#FP
FlappyParrot	1410	58.66	1.0	1410	61.66	1.0
Frogger	6110	95.63	3.00	6045	95.86	2.78
Fruitcatching	2549	90.25	13.44	2544	91.58	15.00
CatchTheDots	3734	84.65	7.22	3745	91.80	14.29
CatchTheGifts	3334	85.47	69.51	3331	87.14	59.74
EndlessRunner	6690	57.05	2.66	6690	60.94	12.00
CatchingApples	1290	79.89	35.33	1290	83.96	35.66
FallingStars	5520	92.50	29.27	5529	93.31	26.51
SpaceOdyssey	3780	64.16	26.33	3792	66.70	29.66
SnowballFight	1830	63.83	3.33	1830	77.92	3.00
DieZauberlehrlinge	4959	38.89	1.00	4955	43.17	3.66
Dodgeball	3108	95.12	29.48	3102	96.07	23.40
FinalFight	8533	69.14	45.68	8498	66.40	36.25
WhackAMole	8957	90.83	72.07	9006	87.50	58.40
BrainGame	2632	51.76	20.00	2633	49.18	15.33
DesertRally	6960	100	11.66	6974	99.92	9.33
OceanCleanup	4800	91.30	62.33	4800	91.03	62.00
Pong	390	68.22	27.66	390	64.44	28.92
RioShootout	4701	43.48	4.00	4710	41.17	3.00
Dragons	7200	40.17	5.16	7295	33.91	5.35
HackAttack	2842	83.68	17.37	2847	80.59	15.07
LineUp	2340	72.95	42.00	2340	64.53	35.33
BirdsShooter	2907	92.02	14.81	2907	90.56	16.22
Snake	3015	59.60	10.0	3017	48.93	2.66
CityDefender	3112	45.09	57.7	3162	45.72	1.69
Mean	4108.12	72.57	22.40	4113.68	72.15	20.65

be achieved without enforcing both conditional paths.

RQ1: Test Coverage Comparison: N_{BC} vs. N_{SC}

N_{BC} provides better branch and statement coverage in structurally intricate games, while N_{SC} is more efficient for straightforward control flows.

5.2 RQ2: Mutation Analysis for Test Oracle Effectiveness

To evaluate the reliability of N_{BC} and N_{SC} as test oracles, we conducted a mutation analysis on 205,545 mutants across 25 games using eight different mutation operators. The objective was to assess how well each method differentiates between true faults and randomized program behavior, considering both mutation kill scores and false positive rates.

The results, summarized in Table 5, provide insight into the precision of each approach. Figure 4 shows average mutation scores for 30 runs of all case studies combined with respect to each mutation operator for N_{BC} and N_{SC} .

The average mutation score for N_{SC} was 72.57%, slightly higher than N_{BC} 's 72.15%. However, N_{SC} also exhibited a higher false positive rate of 22.40%, whereas N_{BC} maintained a lower false positive rate of 20.65%. While the difference in mean mutation scores is small (0.42%), the difference in false positives (1.75%) suggests that N_{SC} tends to over-detect faults. This implies that N_{SC} often misclassifies normal program variations as faults, making it less precise. Conversely, N_{BC} , with its lower false positive rate, demonstrates a more refined ability to detect actual faults while reducing unnecessary false alarms.

Case 1: N_{BC} Detects More Faults with Fewer False Positives

N_{BC} showed superior performance in several games where decision-making logic played a significant role in the program's execution (as shown in Table 5). These include FlappyParrot, Frogger, Fruit-Catching, EndlessRunner, CatchTheDots, CatchTheGifts, CatchingApples, FallingStars, SpaceOdyssey, SnowballFight, DieZauberlehrlinge, and Dodgeball.

In these cases, N_{BC} successfully identified a greater number of true faults while maintaining a lower rate of misclassification. The results indicate that optimizing for branch coverage allows N_{BC} to evolve networks that are more effective at distinguishing between faulty and non-faulty behavior. This suggests that test cases generated with a branch coverage objective are better suited for exposing control-dependent errors, particularly in decision-heavy programs.

Case 2: N_{SC} Appears to Outperform N_{BC} , due to False Positives

In contrast, there were also a few cases where N_{SC} appeared to outperform N_{BC} in mutation detection (as shown in Table 5), particularly in LineUp, BrainGame, Snake, CityDefender, DesertRally, FinalFight, OceanCleanup, Pong, RioShootout, Dragons, WhackAMole, HackAttack, and BirdsShooter.

However, closer examination of the data reveals that N_{SC} 's higher mutation kill scores were largely driven by its tendency to generate more false positives, rather than detecting a greater number of real faults. This means that while N_{SC} may appear to be detecting more faults, it is in fact failing to differentiate between expected variations in randomized program execution and actual software defects.

For instance, in Pong and RioShootout, N_{SC} achieved a higher kill score, but this was due to misclassifying random execution variations as genuine faults, inflating its overall score. In FinalFight, N_{SC} detected a slightly greater number of faults, but also misclassified a significantly larger number of non-faulty behaviors, further demonstrating its tendency to over-predict errors.

Case 3: Performance Across Mutation Operators

Further analysis of mutation operators shows that N_{BC} consistently produced lower standard deviations (10%) compared to N_{SC} (30%), indicating a more stable and reliable fault detection mechanism (as depicted in Figure 4). In Arithmetic Operator Replacement (AOR), both methods achieved a 100% mutation kill rate, but N_{BC} 's higher node density suggests a more stable execution process.

N_{BC} also demonstrated greater effectiveness in detecting faults in decision structures, particularly in Negate Conditional Mutation (NCM) and Relational Operator Replacement (ROR), where it outperformed N_{SC} with higher standard deviations of 91% and 71%, respectively. In Single Block Deletion (SBD) and Script Deletion Mutation (SDM), N_{BC} similarly achieved superior results.

However, Variable Replacement Mutation (VRM) posed a challenge for N_{BC} , as its mutation score was 83% with higher density, indicating that variable mutations do not always impact control structures unless they alter decision-making processes. This suggests that N_{SC} , which focuses on executing every statement at least once, may have an advantage in detecting faults in non-decision variables, even though it lacks N_{BC} 's precision in identifying control-flow-related faults.

Insights from from Mutation Analysis

The results of this analysis highlight that while N_{SC} may achieve slightly higher overall mutation kill scores, this advantage is offset by its higher false positive rate, making it less reliable as a test oracle. N_{BC} , despite marginally lower kill scores, proves to be a more precise method for mutation testing, as it effectively reduces the likelihood of misidentifying non-faulty variations as defects.

The inherent randomness of Scratch games further amplifies N_{SC} 's shortcomings, as it frequently mislabels randomized behaviors as faults, leading to misleading conclusions. While N_{SC} may still be useful in detecting a broad range of faults, its higher false positive rate reduces confidence in its results,

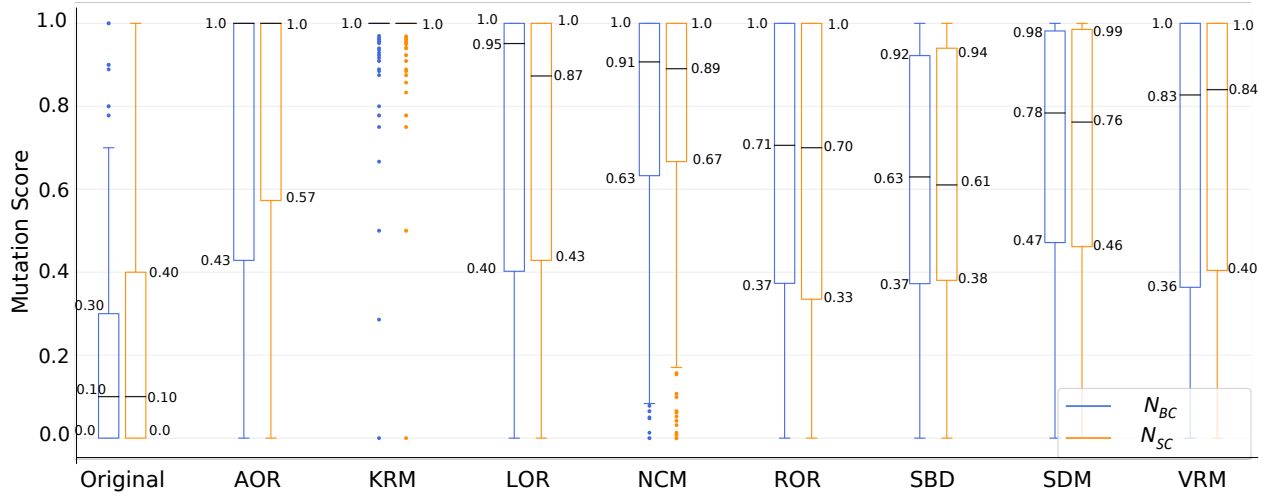


Figure 4. Average mutation scores for 30 runs of all case studies combined with respect to each mutation operator for N_{BC} and N_{SC}

making N_{BC} the preferred choice for scenarios where accuracy and fault differentiation are critical.

Ultimately, N_{BC} emerges as the more robust and precise approach for fault detection in Scratch programs. Its ability to prioritize branch exploration and distinguish real faults from expected program behavior makes it a better-suited testing strategy for complex, decision-based Scratch applications.

RQ2: Test Oracle Effectiveness

N_{BC} is a more robust and precise test oracle, especially for complex, decision-based Scratch applications, as it effectively reduces false positives while maintaining strong fault detection capabilities.

5.3 Discussion and Analysis

The empirical results presented in subsections 5.1 and 5.2 provide key insights into the performance differences between Neatest with Statement Coverage (N_{SC}) and Neatest with Branch Coverage (N_{BC}). This subsection discusses the implications of these findings, analyzes the strengths and weaknesses of each approach, and outlines directions for future research.

5.3.1 Coverage Performance and Structural Impact

Our findings demonstrate that N_{BC} achieves higher mean branch coverage (86.31%) compared to N_{SC} (84.30%), but the degree of improvement varies significantly depending on the structural complexity of the Scratch program. Games with rich control dependencies (e.g., conditional blocks with multiple decision paths) benefited the most from N_{BC} . This is because N_{BC} actively prioritizes branch execution, ensuring that both the true and false paths of conditions are exercised, leading to better fault detection. Conversely, games with mutually exclusive branches (i.e., control structures that inherently cancel each other out) showed no substantial improvement with N_{BC} over N_{SC} . In such cases, statement execution was sufficient to achieve high coverage without the need for explicit branch targeting. This suggests that the effectiveness of N_{BC} is highly dependent on program structure, meaning that blindly applying branch coverage as a default metric may not always lead to optimal test generation results.

5.3.2 Test Oracle Effectiveness: Mutation Kill Score vs. False Positives

When assessing the ability of test cases to act as oracles, we found that while N_{SC} achieved a slightly higher mutation kill score (72.57%) compared to N_{BC} (72.15%), it also exhibited a significantly higher false positive rate (22.40%) compared to N_{BC} 's 20.65%.

N_{BC} 's lower false positive rate suggests that it is better at distinguishing between true faults and random program behavior. N_{SC} , while achieving higher overall kill scores, often misclassified correct behaviors as faults, leading to an inflation of its mutation detection results. This highlights an important trade-off: N_{BC} provides a more precise fault detection mechanism, whereas N_{SC} may produce misleadingly high kill scores due to over-sensitive fault detection.

5.3.3 Impact of Randomness in Scratch Games

One key observation from our experiments is that randomness in game behavior significantly influences test generation effectiveness.

For games with high randomness (e.g., FallingStars, CityDefender, and Snake), neither N_{SC} nor N_{BC} was able to consistently outperform the other. In such games, the inherent variability in program execution makes it difficult to optimize test cases, since the same sequence of inputs may lead to different program states. This raises an important consideration: test generation strategies for highly stochastic environments may require additional mechanisms, such as reinforcement learning or adaptive heuristics, to improve reliability.

5.3.4 Practical Implications for Automated Game Testing

From a practical standpoint, our results suggest that branch coverage should not always be treated as the default metric for game testing.

For structurally simple games, N_{SC} is often sufficient to achieve high test coverage with lower computational overhead. For complex games involving decision trees, event-driven behaviors, or nested control structures, N_{BC} provides a meaningful advantage by ensuring that all possible execution paths are explored. Test oracle effectiveness is critical— N_{BC} 's lower false positive rate makes it more suitable for applications where distinguishing actual faults from random behavior is essential. Future work should explore hybrid approaches, where test generation dynamically switches between statement and branch coverage strategies based on program complexity. Additionally, context-aware fitness functions could be integrated to adaptively adjust search criteria based on game state variability.

Our results suggest that N_{BC} generally outperforms N_{SC} for complex Scratch programs, particularly when full logical path coverage is required. Additionally, N_{BC} provides more precise fault detection with fewer false positives. Future research should focus on developing hybrid and adaptive test generation approaches to further improve automated game testing for Scratch programs.

6 THREATS TO VALIDITY

This section outlines the potential threats to validity in our study, categorized into external validity, internal validity, and construct validity.

6.1 External Validity

External validity concerns the generalizability of our findings beyond the specific dataset used in this study. Our dataset includes 25 diverse Scratch games, carefully selected to represent a wide range of game structures, interaction mechanisms, and control dependencies. However, we cannot guarantee that our results generalize to all Scratch applications, particularly large-scale commercial projects or games developed in different programming languages. Expanding the dataset in future studies to include a

broader variety of Scratch projects and non-game applications could enhance the generalizability of our findings.

6.2 Internal Validity

Internal validity relates to factors that could affect the accuracy and reliability of our experimental results. One key limitation is the randomized nature of the experiments, which may cause slight variations in results across different runs. We mitigated this threat by performing 30 repetitions for RQ1 (test coverage evaluation) and 100 repetitions for RQ2 (mutation analysis) to ensure statistical robustness.

Additionally, our experiments used a fixed search budget of 5 hours per test case. While this was sufficient for most Scratch games, highly complex games with extensive control dependencies may require longer execution times for full test suite optimization. Dynamic search budgets, based on game complexity, could be explored in future work to improve coverage for more complex Scratch applications.

Furthermore, the inherent randomness in Scratch game execution posed challenges, particularly for games with stochastic behaviors. Although we accounted for this by averaging results across multiple runs, adaptive testing strategies that dynamically adjust search techniques based on program state variability could further enhance test generation effectiveness.

6.3 Construct Validity

Construct validity pertains to whether our study accurately measures the intended concepts. In our evaluation, we used branch coverage, statement coverage, and mutation analysis as primary metrics. While these metrics are widely accepted in software testing research, they may not fully capture all dimensions of test effectiveness, such as real-world bug detection capabilities.

Additionally, the false positives observed in mutation analysis could slightly skew the perceived effectiveness of N_{SC} and N_{BC} as test oracles. N_{SC} 's higher mutation kill scores were sometimes due to over-sensitive fault detection, leading to more false positives. Future work could incorporate additional evaluation criteria, such as developer feedback on test case utility or real-world bug detection studies.

7 CONCLUSION

Automated test generation for game-like programs presents unique challenges due to their intentional complexity, dynamic user interactions, and inherent randomness. Traditional static test generators struggle to effectively test these programs as they fail to differentiate between novel and erroneous behaviors. This limitation prevents them from achieving comprehensive test coverage and robust fault detection. To address these challenges, this study introduced a novel test generator with a branch coverage-focused fitness function, extending the capabilities of Neatest for Scratch game testing.

Our approach leverages neuroevolution-based dynamic test suites, where neural networks evolve to reliably reach program states, including winning conditions, despite randomized execution paths. By incorporating branch coverage optimization, our method ensures higher test coverage, overcoming the limitations of previous test generators that only optimized for statement coverage. Empirical evaluation across 25 diverse Scratch games demonstrates that our branch-coverage-driven approach achieves higher test coverage ratios and improved fault detection capabilities compared to statement coverage-based test generation. Additionally, the networks function as test oracles, leveraging their structure to detect unexpected behaviors in mutant programs. The results show that Neatest with Branch Coverage (N_{BC}) consistently achieves more reliable test coverage while reducing false positives, making it a more effective approach for automated game testing.

While our approach significantly improves coverage and fault detection, several areas remain open for future research. Expanding the dataset beyond the 25 Scratch games used in this study, including commercial Scratch projects and non-game educational applications, would provide broader validation of our method. Another key direction is adaptive search strategies, as our fixed search budget of five hours per test case may not be optimal for highly complex games. Future research should explore dynamic resource allocation based on game complexity and execution behavior. Additionally, handling non-deterministic behaviors remains a challenge, as games with highly randomized execution paths require more advanced test generation techniques. Integrating reinforcement learning-based strategies or adaptive heuristics could further enhance test generation stability in stochastic environments.

Furthermore, our current approach focuses solely on branch coverage, but future work should explore hybrid test generation techniques that dynamically switch between branch and statement coverage based on program structure and testing needs. This could further improve test efficiency and fault detection rates. Another promising research avenue is real-world bug detection and developer feedback integration, where the effectiveness of N_{BC} -generated test cases is assessed based on actual defect discovery in Scratch programs. Finally, integrating our method into educational and industry-level game testing frameworks could provide practical validation and real-world applicability, especially for large-scale game development environments like Unity or Godot.

In summary, this research advances automated game testing by enhancing coverage criteria, fault detection, and test oracle effectiveness. By leveraging neuroevolution and dynamic test suites, our approach bridges the gap between automated test generation and human-like gameplay interactions. While N_{BC} clearly outperforms statement coverage-based methods, further research on adaptive search strategies, stochastic behavior handling, and real-world applicability will further solidify its role as a robust automated testing framework for game-like programs.

REFERENCES

- [1] Statista. Games - worldwide. <https://www.statista.com/outlook/amo/media/games/worldwide?currency=USD>, 2025. Accessed: 2025-05-02.
- [2] Adina Deiner, Patric Feldmeier, Gordon Fraser, Sebastian Schweikl, and Wengran Wang. Automated test generation for scratch programs. *Empirical Software Engineering*, 28(3):79, 2023.
- [3] Cristiano Politowski, Fabio Petrillo, and Yann-Gaël Guéhéneuc. A survey of video game testing. In *2021 IEEE/ACM International Conference on Automation of Software Test (AST)*, pages 90–99. IEEE, 2021.
- [4] Norizan Mat Diah, Marina Ismail, Suzana Ahmad, and Mohd Khairulnizam Md Dahari. Usability testing for educational computer game using observation method. In *2010 international conference on information retrieval & knowledge management (CAMP)*, pages 157–161. IEEE, 2010.
- [5] Xavier Ferre, Angelica de Antonio, Ricardo Imbert, and Nelson Medinilla. Playability testing of web-based sport games with older children and teenagers. In *Human-Computer Interaction. Interacting in Various Application Domains: 13th International Conference, HCI International 2009, San Diego, CA, USA, July 19-24, 2009, Proceedings, Part IV 13*, pages 315–324. Springer, 2009.
- [6] Chang-Sik Cho, Kang-Min Sohn, Chang-Jun Park, and Ji-Hoon Kang. Online game testing using scenario-based control of massive virtual users. In *2010 The 12th International Conference on Advanced Communication Technology (ICACT)*, volume 2, pages 1676–1680. IEEE, 2010.
- [7] Christopher Schaefer, Hyunsook Do, and Brian M Slator. Crushinator: A framework towards game-

- independent testing. In 2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE), pages 726–729. IEEE, 2013.
- [8] Adam Smith, Mark Nelson, and Michael Mateas. Computational support for play testing game sketches. In Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, volume 5, pages 167–172, 2009.
 - [9] Sinan Ariyurek, Aysu Betin-Can, and Elif Surer. Automated video game testing using synthetic and humanlike agents. IEEE Transactions on Games, 13(1):50–67, 2019.
 - [10] Johannes Pfau, Jan David Smeddinck, and Rainer Malaka. Automated game testing with icarus: Intelligent completion of adventure riddles via unsupervised solving. In Extended abstracts publication of the annual symposium on computer-human interaction in play, pages 153–164, 2017.
 - [11] Cristiano Politowski, Yann-Gaël Guéhéneuc, and Fabio Petrillo. Towards automated video game testing: still a long way to go. In Proceedings of the 6th international ICSE workshop on games and software engineering: engineering fun, inspiration, and motivation, pages 37–43, 2022.
 - [12] Feng Xue, Junsheng Wu, and Tao Zhang. Learning-replay based automated robotic testing for mobile app. Mobile Information Systems, 2022(1):1084602, 2022.
 - [13] Patric Feldmeier and Gordon Fraser. Learning by viewing: Generating test inputs for games by integrating human gameplay traces in neuroevolution. In Proceedings of the Genetic and Evolutionary Computation Conference, pages 1508–1517, 2023.
 - [14] John Maloney, Mitchel Resnick, Natalie Rusk, Brian Silverman, and Evelyn Eastmond. The scratch programming language and environment. ACM Transactions on Computing Education (TOCE), 10(4):1–15, 2010.
 - [15] Andreas Stahlbauer, Marvin Kreis, and Gordon Fraser. Testing scratch programs automatically. In Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pages 165–175, 2019.
 - [16] Patric Feldmeier and Gordon Fraser. Neuroevolution-based generation of tests and oracles for games. In Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, pages 1–13, 2022.
 - [17] Kenneth O Stanley and Risto Miikkulainen. Evolving neural networks through augmenting topologies. Evolutionary computation, 10(2):99–127, 2002.
 - [18] Gordon Fraser and Andrea Arcuri. Whole test suite generation. IEEE Transactions on Software Engineering, 39(2):276–291, 2012.
 - [19] Simeon C. Ntafos and S. Louis Hakimi. On path cover problems in digraphs and applications to program testing. IEEE Transactions on Software Engineering, (5):520–529, 1979.
 - [20] Cheer-Sun D Yang, Amie L Souter, and Lori L Pollock. All-du-path coverage for parallel programs. ACM SIGSOFT Software Engineering Notes, 23(2):153–162, 1998.
 - [21] Dorothy Graham, Erik van Veenendaal, Isabel Evans, and Rex Black. Foundations of software testing: ISTQB certification. Intl Thomson Business Pr, 2008.
 - [22] Hadi Hemmati. How effective are code coverage criteria? In 2015 IEEE International Conference on Software Quality, Reliability and Security, pages 151–156. IEEE, 2015.
 - [23] Hong Zhu, Patrick AV Hall, and John HR May. Software unit test coverage and adequacy. Acm computing surveys (csur), 29(4):366–427, 1997.
 - [24] Phil McMinn. Search-based software test data generation: a survey. Software testing, Verification and reliability, 14(2):105–156, 2004.
 - [25] Joachim Wegener, André Baresel, and Harmen Sthamer. Evolutionary test environment for automatic structural testing. Information and software technology, 43(14):841–854, 2001.

- [26] Bogdan Korel. Automated software test data generation. IEEE Transactions on software engineering, 16(8):870–879, 1990.
- [27] Luciano Baresi, Pier Luca Lanzi, and Matteo Miraz. Testful: an evolutionary test approach for java. In 2010 Third International Conference on Software Testing, Verification and Validation, pages 185–194. IEEE, 2010.
- [28] Gordon Fraser and Andrea Arcuri. Evosuite: automatic test suite generation for object-oriented software. In Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering, pages 416–419, 2011.
- [29] Paolo Tonella. Evolutionary testing of classes. ACM SIGSOFT Software Engineering Notes, 29(4):119–128, 2004.
- [30] Andrea Arcuri. Restful api automated test case generation with evomaster. ACM Transactions on Software Engineering and Methodology (TOSEM), 28(1):1–37, 2019.
- [31] Florian Gross, Gordon Fraser, and Andreas Zeller. Search-based system testing: high coverage, no false alarms. In Proceedings of the 2012 International Symposium on Software Testing and Analysis, pages 67–77, 2012.
- [32] Ke Mao, Mark Harman, and Yue Jia. Sapienz: Multi-objective automated testing for android applications. In Proceedings of the 25th international symposium on software testing and analysis, pages 94–105, 2016.
- [33] Leon Sell, Michael Auer, Christoph Frädrieh, Michael Gruber, Philemon Werli, and Gordon Fraser. An empirical evaluation of search algorithms for app testing. In Testing Software and Systems: 31st IFIP WG 6.1 International Conference, ICTSS 2019, Paris, France, October 15–17, 2019, Proceedings 31, pages 123–139. Springer, 2019.
- [34] Ron Cytron, Jeanne Ferrante, Barry K Rosen, Mark N Wegman, and F Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. ACM Transactions on Programming Languages and Systems (TOPLAS), 13(4):451–490, 1991.
- [35] Adina Deiner, Christoph Frädrieh, Gordon Fraser, Sophia Geserer, and Niklas Zantner. Search-based testing for scratch programs. In Search-Based Software Engineering: 12th International Symposium, SSBSE 2020, Bari, Italy, October 7–8, 2020, Proceedings 12, pages 58–72. Springer, 2020.
- [36] Katharina Götz, Patric Feldmeier, and Gordon Fraser. Model-based testing of scratch programs. In 2022 IEEE Conference on Software Testing, Verification and Validation (ICST), pages 411–421. IEEE, 2022.
- [37] Patric Feldmeier and Gordon Fraser. Combining neuroevolution with the search for novelty to improve the generation of test inputs for games. In Proceedings of the 1st ACM International Workshop on Foundations of Applied Software Engineering for Games, pages 14–19, 2024.
- [38] Patric Feldmeier, Katrin Schmelz, and Gordon Fraser. Many-objective neuroevolution for testing games. arXiv preprint arXiv:2501.07954, 2025.
- [39] Andrea Arcuri and Lionel Briand. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In Proceedings of the 33rd international conference on software engineering, pages 1–10, 2011.
- [40] András Vargha and Harold D Delaney. A critique and improvement of the cl common language effect size statistics of mcgraw and wong. Journal of Educational and Behavioral Statistics, 25(2):101–132, 2000.
- [41] Henry B Mann and Donald R Whitney. On a test of whether one of two random variables is stochastically larger than the other. The annals of mathematical statistics, pages 50–60, 1947.
- [42] A Jefferson Offutt, Ammei Lee, Gregg Rothermel, Roland H Untch, and Christian Zapf. An exper-

imental determination of sufficient mutant operators. ACM Transactions on Software Engineering and Methodology (TOSEM), 5(2):99–118, 1996.

- [43] Jinhan Kim, Robert Feldt, and Shin Yoo. Guiding deep learning system testing using surprise adequacy. In 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), pages 1039–1049. IEEE, 2019.
- [44] Richard A Davis, Keh-Shin Lii, and Dimitris N Politis. Remarks on some nonparametric estimates of a density function. In Selected Works of Murray Rosenblatt, pages 95–100. Springer, 2011.

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to [Patric Feldmeier](#) for his invaluable support during the execution and implementation setup of our approach. His previous work in the development of the NEATest algorithm served as a key foundation for our work. We greatly appreciate his contributions and collaboration throughout this research.

AUTHOR CONTRIBUTIONS

Conceptualization: Khizra Sohail; **Methodology:** Khizra Sohail, Atif Aftab Ahmed Jilani; **Validation:** Khizra Sohail; **Result analysis:** Khizra Sohail, Nigar Azhar Butt; **Visualization:** Khizra Sohail, Nigar Azhar Butt; **Supervision:** Atif Aftab Ahmed Jilani; **Writing – original draft:** Khizra Sohail, Nigar Azhar Butt; **Writing – review & editing:** Khizra Sohail, Nigar Azhar Butt, Atif Aftab Jilani.

DATA AVAILABILITY

The data for framework is available at <https://github.com/khizra777/Neatest-Branch>.

The data for results is available at <https://github.com/khizra777/NEATEST-Branch-Evaluation>.

FUNDING

The authors report that there are no funding details to declare.

COMPETING INTERESTS

The authors declare that they have no competing interests related to this study.